

Retell AI — Topic 6

Custom LLM Integration with Google Gemini

Assessment & Implementation Evidence

Project	Trainee_Sarah_Topic6_CustomLLM
LLM Provider	Google Gemini
Model	gemini-2.5-flash
Integration	Retell Custom LLM + WebSocket + ngrok
Repository	github.com/shaikshahid777/retell-ai-topic-6-custom-llm
Loom	loom.com/share/572e0af396d74336b279f9ad8ef74464

1. Objective

Implement a Retell AI Custom LLM integration using a public WebSocket endpoint, authenticate the connection with a query parameter, process Retell interaction events, send the conversation to an external LLM, and return a response to Retell.

2. Implemented Architecture

Retell AI Agent → **wss:// public endpoint** → **ngrok** → **Node.js WebSocket server** → **Google Gemini API** → **Node.js response** → **Retell AI**

The server implements URL-query authentication, handles **response_required** and **update_only** events, builds conversation context, calls Gemini, and returns Retell-compatible JSON responses. Goodbye handling includes an **end_call: true** payload.

3. Environment & Security

The Gemini API key is stored in the local **.env** file and is not hard-coded into server.js. The project uses a custom WebSocket authentication token supplied through the environment. The **.env** file is excluded from the repository.

4. Validation Evidence

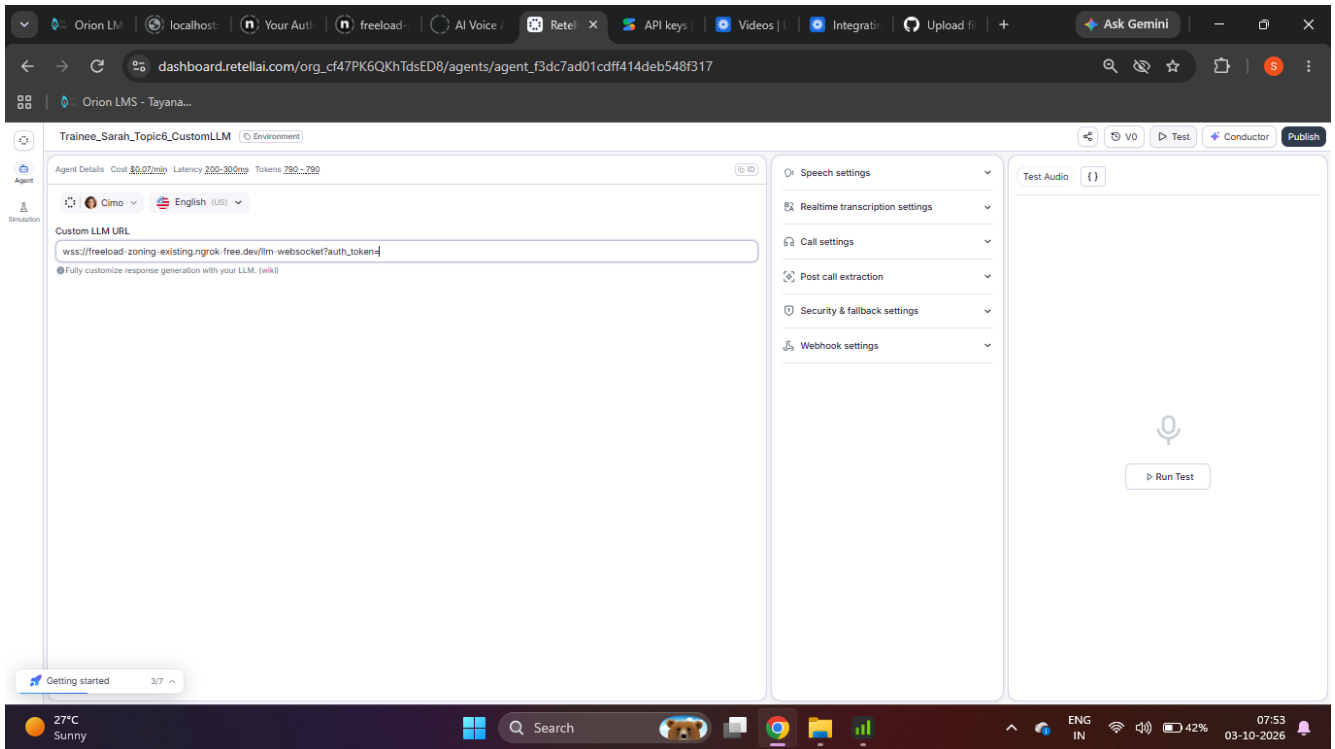


Figure 1 — Retell Custom LLM agent configuration

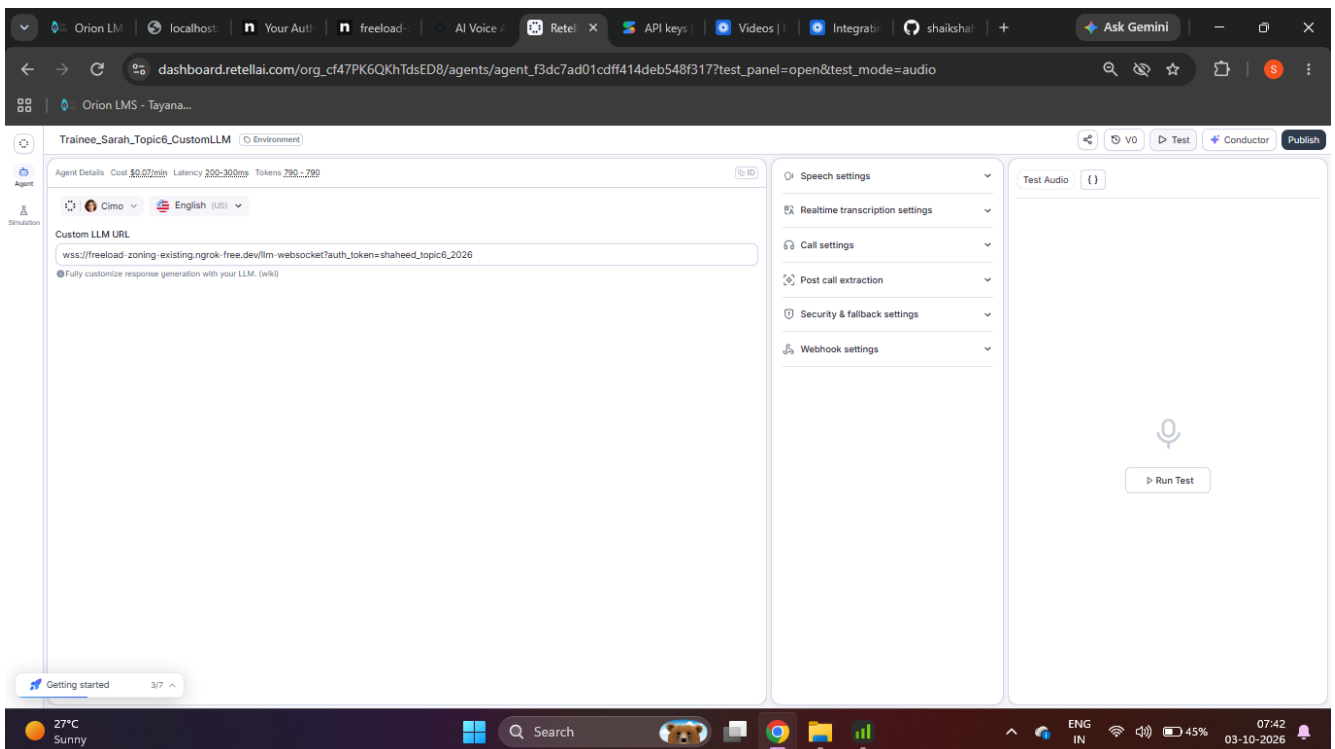


Figure 2 — Node.js WebSocket server logs

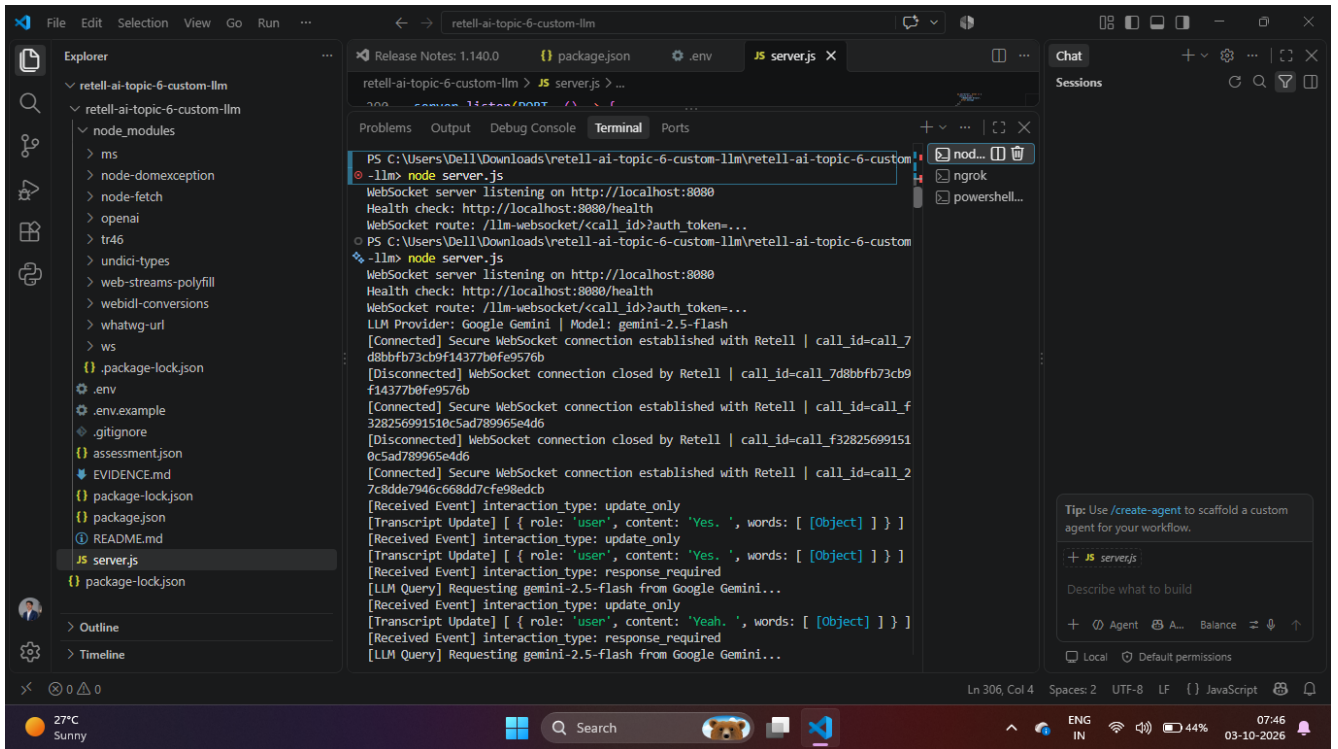


Figure 3 — ngrok public WebSocket tunnel

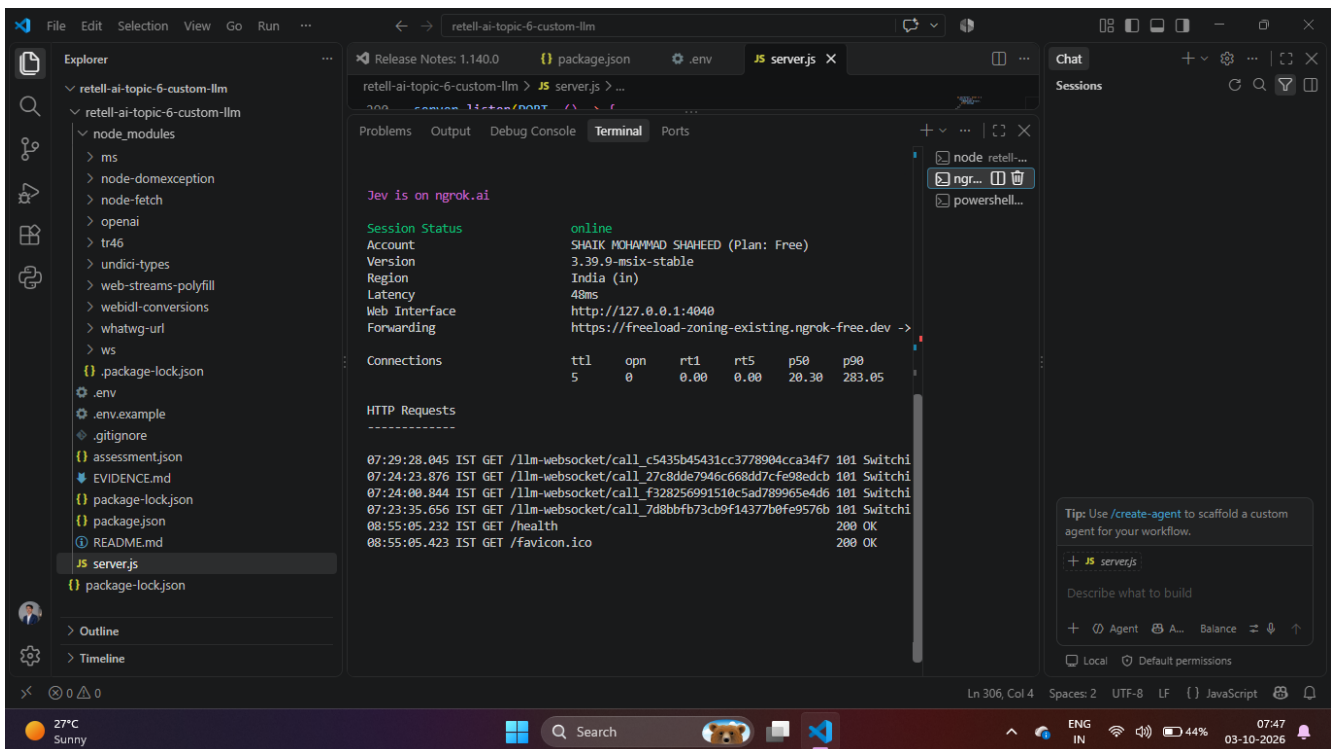


Figure 4 — Gemini API verification

5. Retell Test Evidence

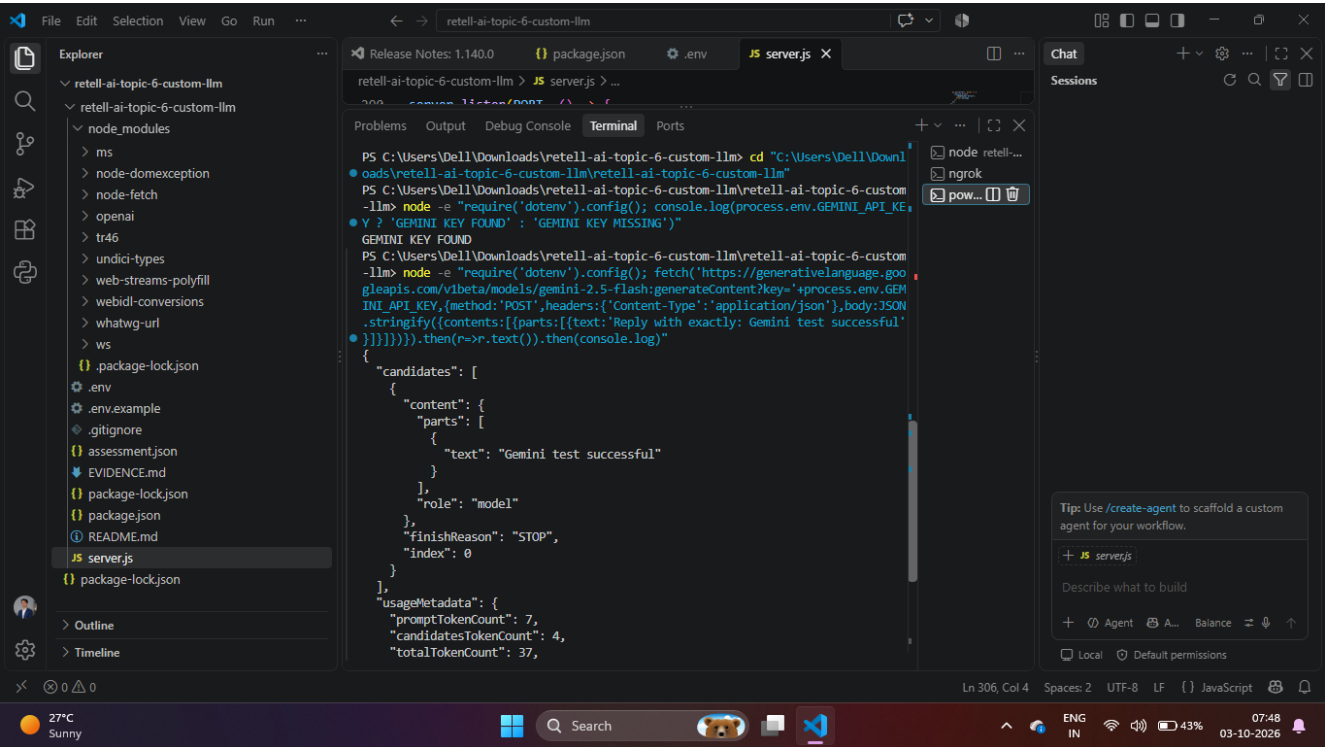


Figure 5 — Retell call history and transcript

6. Test Result & Scope Note

The Gemini API was independently verified successfully, and the Retell-to-WebSocket connection produced server-side WebSocket activity including response_required events. However, the final Retell Playground call was recorded as **Unsuccessful** in Retell Call History. Therefore, this document does not claim a fully successful end-to-end voice test. The evidence demonstrates the implemented server, public WebSocket connection, Gemini API availability, and Retell event processing observed during testing.

7. Key Evidence Observed

Evidence	Observed result
Local server	Node.js WebSocket server listening on port 8080
Gemini	gemini-2.5-flash API request returned “Gemini test successful”
ngrok	Public HTTPS/WSS forwarding active
WebSocket	Retell connections returned HTTP 101 Switching Protocols
Retell events	update_only and response_required events observed in server logs
Call history	Retell stored the test conversation and marked the final call Unsuccessful

8. Deliverables

- Source code repository: github.com/shaikshahid777/retell-ai-topic-6-custom-llm
- Loom demonstration: loom.com/share/572e0af396d74336b279f9ad8ef74464
- Screenshots: included in this evidence document
- Local environment: Node.js, WebSocket server, ngrok and Google Gemini API

Security reminder: API keys and authentication secrets are intentionally not included in this document. Any previously exposed authentication token should be rotated after submission.